Testaufgabe Junior PHP Developer (m/w/d)

PAPSTAR

Hallo und danke für dein Interesse an der Stelle bei PAPSTAR.

Im nächsten Gespräch wollen wir mit dir über echten Code reden statt über abstrakte Fragen. Dafür haben wir eine kleine Aufgabe vorbereitet. Plane bitte **nicht mehr als drei Stunden** ein. Uns geht es nicht um Vollständigkeit. Wir wollen sehen, wie du an ein Problem herangehst.

Aufgabe A: Artikelimport (Pflichtteil)

Du bekommst die Datei `artikel.csv` mit Artikeldaten aus einem Fremdsystem. Wie im Alltag meistens ist sie nicht sauber. Schreibe ein PHP-Skript für die Kommandozeile, das diese Datei einliest, prüft, aufräumt und als JSON ausgibt.

Aufruf:

```
php import.php artikel.csv ausgabe.json
```

Die Datei hat die Spalten `sku`, `name`, `preis`, `bestand`, `kategorie` und `ean`.

Das soll dein Skript tun

- Datensätze ohne SKU oder mit einem Preis, der sich nicht auswerten lässt, werden nicht importiert. Sie kommen stattdessen in ein Fehlerprotokoll, mit Zeilennummer und Grund. Die Zeilennummer bezieht sich auf die Datei, die Kopfzeile ist Zeile 1.
- Preise kommen in verschiedenen Formaten vor, zum Beispiel `12,90`, `12.90`, `12,90 €` oder `9,5` . Mach daraus jeweils eine Zahl mit zwei Nachkommastellen.
- Der Bestand ist eine ganze Zahl. Fehlt der Wert, gilt 0. Negative Werte sind ungültig.
- Kommt dieselbe SKU mehrfach vor, gewinnt der letzte Datensatz in der Datei.
- Die Ausgabe ist ein JSON-Array, sortiert nach SKU.
- Am Ende gibt das Skript auf der Konsole eine kurze Zusammenfassung aus: wie viele Zeilen es gelesen, importiert und übersprungen hat.

> **Nicht alles ist eindeutig geregelt.** Das ist Absicht. Wo du eine Annahme triffst, entscheide selbst und schreib sie kurz ins README. Es gibt dafür keine falsche Antwort, solange du sie begründest.

Technischer Rahmen

- PHP 8.1 oder neuer.
- Du brauchst kein Framework, keine Datenbank und keine Oberfläche.
- Externe Bibliotheken darfst du nutzen, musst du aber nicht.

Worauf wir achten

- Der Code ist objektorientiert aufgebaut. Es gibt mindestens eine Klasse mit einer klar benennbaren Aufgabe. Einlesen, Prüfen und Schreiben stehen nicht alle in derselben Methode.
- Ein fehlerhafter Datensatz bringt nicht den ganzen Durchlauf zum Absturz.
- Klassen, Methoden und Variablen haben Namen, die sagen, was sie tun.
- Es gibt eine README-Datei mit drei Punkten: wie man das Skript startet, was du bewusst so und nicht anders gelöst hast (ein bis zwei Entscheidungen genügen) und was du mit mehr Zeit noch gemacht hättest.

Aufgabe B: Magento (freiwillig)

Nur falls du schon mit Magento gearbeitet hast und Lust darauf hast. Wenn du diese Aufgabe weglässt, hat das keinen Einfluss auf die Bewertung. Das ist wirklich so gemeint.

Baue ein minimales Modul `Papstar_ArtikelLabel`, das

- über einen Data Patch das Produktattribut `papstar_label` (Typ Text) anlegt und
- einen CLI-Befehl bereitstellt, mit dem sich dieses Attribut für eine SKU setzen lässt, zum Beispiel:

```
bin/magento papstar:label:set ABC-123 "Neu im Sortiment"
```

Du musst das Modul nicht in einer laufenden Installation testen. Uns interessiert der Aufbau, nicht ob es fehlerfrei durchläuft.

Abgabe und Hilfsmittel

- Schick uns eine ZIP-Datei oder einen Link auf ein Repository an job@papstar.de.
- Du hast eine Woche Zeit. Wenn du länger brauchst, sag einfach Bescheid. Das ist kein Minuspunkt.
- KI-Assistenten darfst du benutzen. Schreib im README bitte kurz dazu, welche. Im Gespräch gehen wir den Code gemeinsam durch, und du erklärst uns deine Lösung.
- Wenn du nicht fertig wirst, gib ab, was du hast, und notiere, was noch fehlt. Das ist besser als gar keine Abgabe.

Anlage

`artikel.csv`: 10 Datensätze. Alle Artikeldaten sind erfunden.

Wir freuen uns auf deine Lösung und das Gespräch darüber.